

O que o agente precisa saber agora — e o que precisa lembrar depois?

Encontro 05 · CampusBot

O CampusBot já consegue agir.

O que acontece se ele esquecer o resultado?

Preveja a segunda decisão

“Veja se há chamado para o projetor do Lab 4. Se não houver, abra um.”

```
consultar_chamados("Lab 4") → []  
próxima decisão → ?
```

Execute a falha

```
uv run python praticas/encontro-05/01_esqueceu_o_resultado.py --offline
```

Encontre o resultado da ferramenta e o estado da rodada seguinte.

O sintoma

```
consulta → []  
resultado descartado  
nova decisão → consultar outra vez
```

Qual linha do terminal prova que a ferramenta funcionou?

Faça a menor alteração

```
atualizar_estado(estado, "consultar_chamados", resultado)
```

```
uv run python praticas/encontro-05/02_estado_entre_passos.py --offline
```

Agora podemos nomear

Estado é o que o sistema mantém nesta execução para usar em uma decisão posterior.

```
consultar → guardar resultado → decidir de novo
```

**Se o sistema guarda mais informação,
tudo deve ir para o modelo?**

Informação demais

```
uv run python praticas/encontro-05/03_contexto_demais.py --offline
```

Quais campos ajudam a decidir se deve abrir um chamado?

Informação de menos

```
uv run python praticas/encontro-05/04_contexto_de_menos.py --offline
```

O que o estado já sabe? O que foi enviado? O que ficou fora?

Dois agentes iguais podem decidir diferente apenas pelo que recebem agora?

Experimento A/B

```
uv run python praticas/encontro-05/05_comparacao_ab.py --offline
```

A → consulta ausente do recorte

B → consulta vazia presente no recorte

Mesmo modelo, prompt, ferramentas, objetivo e estado inicial.

A distinção que o A/B exigiu

estado → tudo que o sistema mantém nesta execução
contexto → recorte entregue ao decisor nesta rodada

**Uma decisão ruim pode vir
da informação que o sistema entregou.**

**E se a informação recebida
era verdadeira ontem?**

Consulta antiga, ambiente novo

```
uv run python praticas/encontro-05/06_informacao_desatualizada.py --offline
```

Preveja: com “nenhum chamado” no estado e CH-001 no ambiente, o que acontece?

Ter uma informação não basta

```
estado: nenhuma ocorrência encontrada antes  
ambiente agora: CH-001 aberto
```

Qual informação serve para esta decisão?

Duas indicações de local

```
estado.local = Lab 4  
nova percepção = "Na verdade, Lab 5."
```

```
uv run python praticas/encontro-05/07_informacao_conflitante.py --offline
```

O que vimos até aqui?

faltou resultado → repete
recorte insuficiente → repete
consulta antiga → duplica
local antigo → age no lugar errado

O programa terminou.

Qual era o protocolo?

Dois processos, um JSON local

```
uv run python praticas/encontro-05/08_memoria_entre_execucoes.py \
  --fase gravar --arquivo /tmp/campusbot-encontro-05.json --offline
```

Depois execute com `--fase recuperar` e o mesmo arquivo.

O que sobreviveu?

```
estado novo → não contém o protocolo  
JSON local → contém CH-001
```

Memória é informação que pode sobreviver além da execução.

Lembrar sempre ajuda?

O status lembrado está velho

```
uv run python praticas/encontro-05/09_memoria_desatualizada.py --offline
```

```
memória: CH-001 aberto  
ambiente: CH-001 encerrado
```

Três perguntas para qualquer decisão

ESTADO 0 que o sistema mantém nesta execução?
CONTEXTO 0 que o decisor recebe agora?
MEMÓRIA 0 que pode sobreviver a outra execução?

Sua investigação continua

```
uv run python praticas/encontro-05/atividade_autonoma.py estado --offline
uv run python praticas/encontro-05/atividade_autonoma.py contexto --offline
uv run python praticas/encontro-05/atividade_autonoma.py memoria --offline
```

Para cada caso: evidência → menor correção → nova execução → explicação.

Se a decisão ficou ruim, como descobrir a causa?

Modelo · ferramenta · estado · contexto · memória

Encontro 06: observar e diagnosticar.